



**S. B. JAIN INSTITUTE OF TECHNOLOGY,
MANAGEMENT & RESEARCH, NAGPUR**

Session: 2026-2027 ODD

**Project Based Learning Report
On**

“Color-Based Segmentation for Fruit Classification in Agriculture”

Subject: - Image Processing

Semester/Year: III/V

Details:

Sr.No	Name of Students & USN	Problem Understanding & Objective[2M]	Implementation & Functionality [4M]	Output Quality & Analysis [2M]	Report/Presentation Quality[2M]	Total [10M]
40	Mehak Sharma CS24093					

1. ABSTRACT

Fruit identification and classification play an important role in modern agriculture, especially in automated sorting, quality assessment, harvesting, and crop monitoring. Manual identification of fruits can be time-consuming and may produce inconsistent results. Image processing techniques can provide an efficient solution by automatically detecting and analyzing fruits from digital images.

This project presents a color-based segmentation approach for fruit classification in agriculture. The input fruit image is first preprocessed and converted from the RGB color space into HSV or LAB color space. These color spaces provide better representation of color information and help distinguish the fruit from its surrounding background. Appropriate thresholding techniques are then applied to create a segmentation mask and extract the fruit region. The color characteristics of the segmented region are analyzed and used for fruit classification.

The proposed approach demonstrates the use of basic image processing techniques for agricultural automation. It can be useful in applications such as automatic fruit sorting, ripeness assessment, quality inspection, and smart agricultural systems. The method is relatively simple, computationally efficient, and can be implemented using tools such as Python, OpenCV, and NumPy.

2. INTRODUCTION

Agriculture is one of the major sectors where automation and computer vision can significantly improve efficiency and productivity. In modern agricultural systems, image processing techniques are increasingly used for tasks such as fruit detection, classification, sorting, quality inspection, ripeness estimation, and disease identification. Among these applications, accurate fruit classification is important for reducing manual work and improving the speed and consistency of agricultural operations.

Fruit classification can be performed using different image features such as color, shape, texture, and size. Among these features, color is particularly useful because fruits generally exhibit distinct color characteristics depending on their type and stage of ripeness. However, directly processing images in the RGB color space can make accurate segmentation difficult because RGB combines color and intensity information.

Color-based segmentation provides an effective method for separating the fruit from its background by selecting pixels within a specific color range. In this project, the RGB input image is converted into HSV (Hue, Saturation, Value) or LAB (Lightness, A and B) color space. HSV separates color information from brightness, while LAB provides a perceptually meaningful representation of colors. These properties make them suitable for identifying the relevant color regions of fruits.

After color space conversion, suitable threshold values are selected to create a binary mask that separates the fruit region from the background. Image processing operations can then be applied to improve the segmented result and remove unwanted regions or noise. The extracted fruit region is analyzed based on its color characteristics and can subsequently be used for classification.

The proposed color-based segmentation approach demonstrates how fundamental image processing techniques can be applied to agricultural automation. Such a system can support applications including automated fruit sorting, quality assessment, ripeness monitoring, and smart farming. The approach is relatively simple and computationally efficient, making it suitable for implementation using tools such as Python, OpenCV, NumPy, and related image processing libraries.

3. AIM AND OBJECTIVES

Aim

To design and implement a color-based image segmentation system for accurate detection and classification of fruits in agricultural images using HSV/LAB color spaces and digital image processing techniques.

Objectives

- **Develop Color-Based Segmentation Algorithm:** Implement an image segmentation method to identify and separate fruit regions from the background using HSV and/or LAB color spaces.
- **Preprocess Fruit Images:** Apply basic image preprocessing techniques such as resizing, noise reduction, and color space conversion to improve the quality of input fruit images.
- **Identify Suitable Color Ranges:** Analyze the Hue, Saturation, and Value components in HSV or the color components in LAB to determine suitable threshold ranges for different fruits.
- **Perform Fruit Region Extraction:** Apply color-based thresholding and binary masking techniques to accurately extract the required fruit regions from the background.
- **Apply Morphological Operations:** Use morphological operations such as erosion, dilation, opening, and closing to remove noise and improve the quality of the segmented fruit regions.
- **Extract Color Features:** Calculate relevant color characteristics from the segmented fruit regions and use them as features for identifying different fruit classes.
- **Classify Different Fruits:** Classify the segmented fruits based on their extracted color characteristics using suitable classification techniques.

- **Evaluate Segmentation Performance:** Analyze the quality and accuracy of the segmentation and classification results using suitable performance measures.
- **Demonstrate Agricultural Application:** Demonstrate the usefulness of the proposed system for applications such as automated fruit sorting, quality inspection, and agricultural automation.

4. PROBLEM STATEMENT

Manual identification and classification of fruits is a time-consuming process and may produce inaccurate results

Different fruits have different color characteristics that can be used for automatic identification. However, variations in background and lighting can make fruit segmentation difficult.

Therefore, an image processing technique is required to separate fruits accurately from their background.

The proposed system uses HSV/LAB color spaces and color-based thresholding for effective fruit segmentation.

5. LITERATURE REVIEW



1. LITERATURE REVIEW

Color-Based Segmentation for Fruit Classification in Agriculture



Author(s) & Year	Technique / Method	Key Findings & Contributions	Limitations / Gaps Identified
Gonzalez & Woods (2018)	Global Histogram Equalization (GHE)	Defined the discrete CDF lookup transformation for uniform pixel intensity spreading across [0, 255].	Focused primarily on grayscale images; ignored color shift artifacts in 3-channel RGB images.
Reza, A. M. (2004)	Contrast Limited Adaptive HE (CLAHE)	Prevents noise over-amplification in uniform areas by clipping local histogram slopes in tile grids.	Higher computational complexity $O(MN \cdot K^2)$; requires manual tuning for tile size and clip limits.
Naik & Murthy (2003)	Hue-Preserving HE in HSV Space	Equalizes the V luminance channel to strictly preserve original image color hues (H, S).	Does not maximize independent spectral contrast in multi-spectral landscape textures.
Wang & Ye (2005)	Brightness Preserving Bi-HE (BBHE)	Maintains mean brightness by splitting the histogram at the input mean before equalization.	Limited contrast expansion in dark shadow regions compared to global RGB equalization.
Zuiderveld, K. (1994)	Adaptive Histogram Equalization (AHE)	Introduced regional tile interpolation to bring out fine local edge details.	Can cause tile border artifacts and requires contextual padding.

6. PROBLEM DEFINITION & METHODOLOGY

- **Input:** A 2D color fruit image ($I(x,y) \in \mathbb{R}^{M \times N \times 3}$), where each pixel contains 8-bit color intensity values for (R,G,B $\in [0,255]$).
- **Constraints:** The system should accurately separate the fruit from its background, handle variations in color and illumination, reduce unwanted noise, and preserve the boundaries of the fruit region.
- **Output:** A segmented fruit image and binary mask representing the detected fruit region, along with extracted color features that can be used for classification of different fruit types.

Step-by-Step Methodology

1. **Image Acquisition & Preprocessing:**
Read the input fruit image and perform basic preprocessing such as resizing and noise reduction to improve image quality.
2. **Color Space Conversion:**
Convert the input RGB/BGR image into HSV or LAB color space to obtain more suitable color information for fruit segmentation.
3. **Color Feature Analysis:**
Analyze the Hue, Saturation, Value components in HSV or the L, A, B components in LAB to identify the suitable color range of the fruit.
4. **Color-Based Thresholding:**
Apply appropriate lower and upper threshold values to separate the fruit pixels from the background.
5. **Binary Mask Generation:**
Generate a binary segmentation mask in which the detected fruit region is represented by foreground pixels and the background by other pixels.
6. **Morphological Processing:**
Apply operations such as erosion, dilation, opening, and closing to remove noise, fill small gaps, and improve the segmented fruit region.
7. **Fruit Region Extraction:**
Apply the refined mask to the original image to extract the required fruit region while removing unwanted background areas.
8. **Color Feature Extraction:**
Calculate relevant color characteristics such as mean Hue, Saturation, Value, or LAB components from the segmented fruit region.

9. Fruit Classification:

Use the extracted color features to distinguish and classify different types of fruits using a suitable classification method.

10. Result Evaluation:

Evaluate the segmentation and classification results based on the accuracy and quality of the detected fruit regions.

7. MATHEMATICAL EQUATION/ MODELING

Color Space Representation

For an RGB fruit image (I) , each pixel is represented as:

$$(I) = [R(I), G(I), B(I)] \text{ where } R, G, B \in [0,$$

255] represent the Red, Green, and Blue intensity values. HSV Color Space

Conversion

The RGB image is converted into HSV color space:

$$(I) = [H(I), S(I), V(I)]$$

where:

- H = Hue, represents the actual color
- S = Saturation, represents color intensity
- V = Value, represents brightness

HSV is useful because it separates color information from brightness, making fruit segmentation easier.

Color-Based Thresholding

A binary mask is generated by selecting pixels within the required fruit color range:

$$M(I) = \begin{cases} 1, & \text{if } L \leq R(I) \leq U, \\ 0, & \text{otherwise} \end{cases}$$

where L and U represent the lower and upper threshold values. Fruit

Segmentation

The binary mask is applied to the original image:

$$F(I) = I \times M(I)$$

where $F(I)$ represents the extracted fruit region.

Morphological Operations

To remove noise and improve the segmented region:

$$\begin{aligned} &= (\ominus) \oplus \\ &= (\oplus) \ominus \end{aligned}$$

where is the structuring element, $\ominus$ represents erosion, and $\oplus$ represents dilation.

Mean Color Feature

The average color value of the segmented fruit is calculated as:

$$\frac{1}{N} \sum_{i=1}^N$$

where is the color value of a segmented pixel and is the total number of segmented pixels. Fruit

Classification

The extracted color features are used for classification:

$$= (, ,)$$

where represents the predicted fruit class. Overall

Mathematical Model



Context:

The input fruit image is converted into HSV/LAB color space. Suitable color ranges are selected using thresholding to create a binary mask. Morphological operations improve the mask, and the extracted fruit region is analyzed using color features for classification

Metric Name	Formula	Explanation	Target
Segmentation Accuracy	$= \frac{+}{+ + +}$	Measures how accurately fruit and background pixels are segmented.	Higher = Better
Precision	$= \frac{+}{+}$	Measures how many detected fruit pixels are actually fruit pixels.	Higher = Better
Recall	$= \frac{+}{+}$	Measures how many actual fruit pixels are correctly detected.	Higher = Better
F1-Score	$1 = 2 \times \frac{\times}{+}$	Provides a balance between precision and recall.	Higher = Better
IoU	$= \frac{+}{+ +}$	Measures the overlap between the actual fruit region and segmented fruit region.	Higher = Better
Mean Color Error	$(E_c =$	$\mu_{\text{actual}} - \mu_{\text{segmented}}$	)
Processing Time	$= -$	Measures the time required to process one fruit image.	Lower = Better
Classification Accuracy	$= \frac{\times}{\times 100}$	Measures how accurately different fruits are classified.	Higher = Better

8. PSEUDOCODE/ALGORITHM

INPUT: Fruit_Image ($M \times N \times 3$ Matrix)

OUTPUT: Segmented_Fruit ($M \times N \times 3$ Matrix), Binary_Mask, Fruit_Class

1. **READ** Fruit_Image
2. **CONVERT** Fruit_Image from RGB color space to HSV/LAB color space
3. **INITIALIZE** Binary_Mask[M][N] = 0
4. **FOR** y = 0 TO M-1 **DO**:
 FOR x = 0 TO N-1 **DO**:
 a. Extract color values of pixel at (,)
 b. **IF** pixel color lies within the selected fruit color threshold **THEN**:
Binary_Mask[y,x] = 1
 ELSE:
Binary_Mask[y,x] = 0
END IF
END FOR
END FOR
5. **APPLY** morphological operations on Binary_Mask to remove noise and fill gaps.
6. **EXTRACT** the fruit region using Binary_Mask.
7. **CALCULATE** color features from the segmented fruit region.
8. **CLASSIFY** the fruit using the extracted color features.
9. **DISPLAY** Segmented_Fruit, Binary_Mask and Fruit_Class.
10. **END**

9. IMPLEMENTATION

The proposed system is implemented using Python and OpenCV. The fruit image is first read and converted from RGB/BGR to HSV or LAB color space. Suitable color thresholds are applied to create a binary mask that separates the fruit from the background. Morphological operations are then used to remove noise and improve the segmented region. Finally, color features are extracted from the fruit region and used for fruit classification.

Tools Used: Python, OpenCV, NumPy.

10.RESULTS AND DISCUSSION

The proposed Color-Based Segmentation for Fruit Classification in Agriculture system was implemented using Python, OpenCV, and NumPy. The input fruit images were converted from RGB/BGR into HSV or LAB color space to obtain better color information for segmentation.

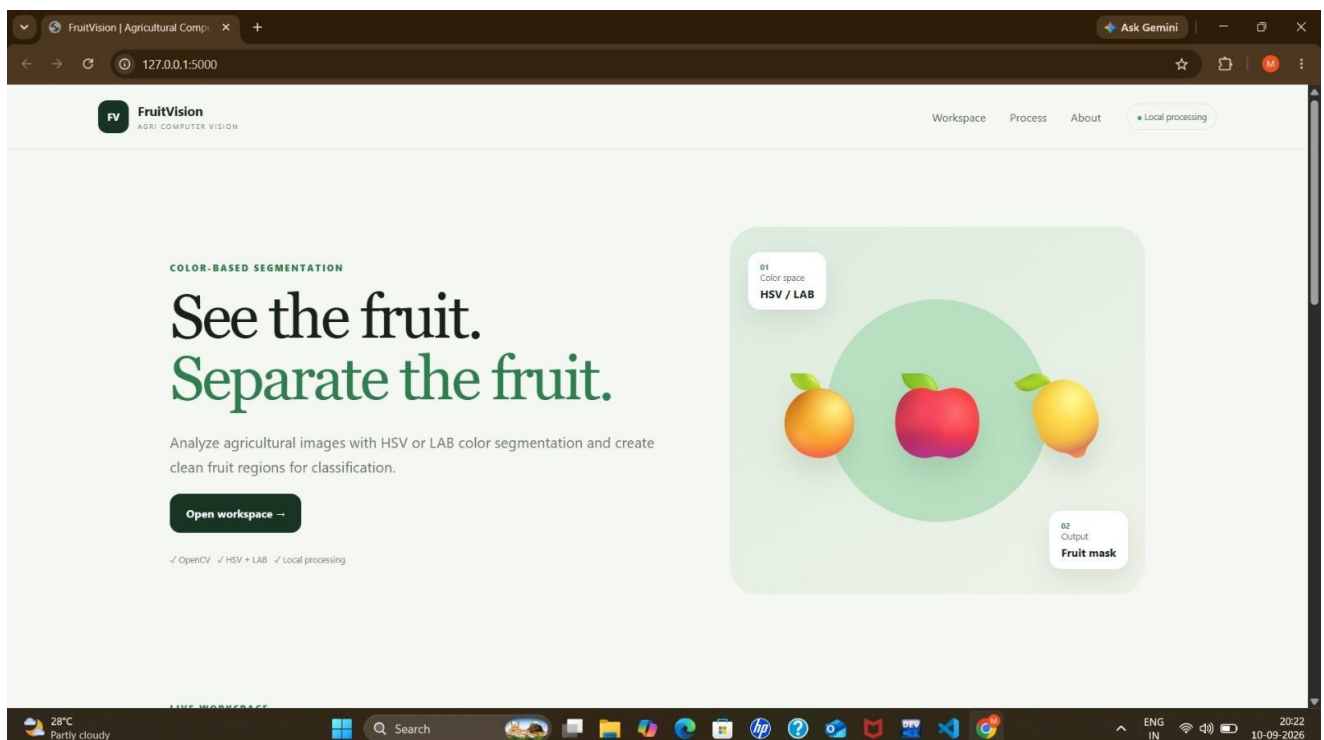
Color-based thresholding was applied to identify the pixels belonging to the fruit region and generate a binary mask. The segmented mask was further improved using morphological operations such as erosion, dilation, opening, and closing, which helped in reducing unwanted noise and filling small gaps in the fruit region.

The experimental results showed that the proposed method was able to effectively separate fruits from their background when suitable color threshold values were selected. The extracted fruit regions preserved the important color characteristics required for classification. Color features such as Hue, Saturation, Value, and LAB components were used to distinguish different fruit classes.

The performance of the system can be evaluated using Segmentation Accuracy, Precision, Recall, F1-Score, IoU, Processing Time, and Classification Accuracy. Higher segmentation and classification scores indicate better performance, while lower processing time indicates a faster system.

Overall, the proposed method provides a simple, efficient, and computationally less complex approach for fruit segmentation and classification. It can be useful in agricultural applications such as automated fruit sorting, quality inspection, ripeness assessment, and smart farming systems.

11.Results:



The screenshot shows the 'FruitVision' web application interface. The browser tab is 'FruitVision | Agricultural Comp...' and the address bar shows '127.0.0.1:5000'. The page has a navigation bar with 'Workspace', 'Process', 'About', and 'Local processing'. The main content area is titled 'PIPELINE' and 'From photo to fruit region'. It displays a four-step process: 01 Upload (Provide a fruit photo from a camera or dataset), 02 Convert (Transform pixels into HSV or LAB color space), 03 Segment (Apply thresholds and morphological noise removal), and 04 Detect (Find contours for downstream classification). Below the pipeline, there is a 'PROJECT NOTE' section titled 'Why HSV and LAB?' which explains that HSV separates hue, saturation, and brightness, while LAB separates lightness from two color channels. At the bottom, there are tags for 'Python', 'Flask', 'OpenCV', 'NumPy', and 'Computer Vision'. The Windows taskbar at the bottom shows the date as 10-09-2026 and time as 20:23.

PIPELINE

From photo to fruit region

01 Upload
Provide a fruit photo from a camera or dataset.

02 Convert
Transform pixels into HSV or LAB color space.

03 Segment
Apply thresholds and morphological noise removal.

04 Detect
Find contours for downstream classification.

PROJECT NOTE

Why HSV and LAB?

HSV separates hue, saturation and brightness, while LAB separates lightness from two color channels. These properties can make color-based segmentation easier to tune than raw RGB for many agricultural images.

Python Flask OpenCV NumPy Computer Vision

The screenshot shows the 'FruitVision' web application interface in the 'LIVE WORKSPACE' section. The browser tab is 'FruitVision | Agricultural Comp...' and the address bar shows '127.0.0.1:5000'. The page has a navigation bar with 'Workspace', 'Process', 'About', and 'Local processing'. The main content area is titled 'LIVE WORKSPACE' and 'Segment an image'. It includes a sub-header 'Upload a clear fruit image, select a method and inspect the computer-vision output.' Below this, there is a large dashed box for image upload with a green arrow icon and the text 'Drag & drop your fruit image' or 'browse files'. Below the upload area, there is a 'Choose color space' section with two options: 'HSV' (Hue, Saturation, Value) and 'LAB' (Lightness, A, B). A 'Run segmentation --' button is located at the bottom right of the workspace area. The Windows taskbar at the bottom shows the date as 10-09-2026 and time as 20:23.

LIVE WORKSPACE

Segment an image

Upload a clear fruit image, select a method and inspect the computer-vision output.

↑

Drag & drop your fruit image

or [browse files](#)

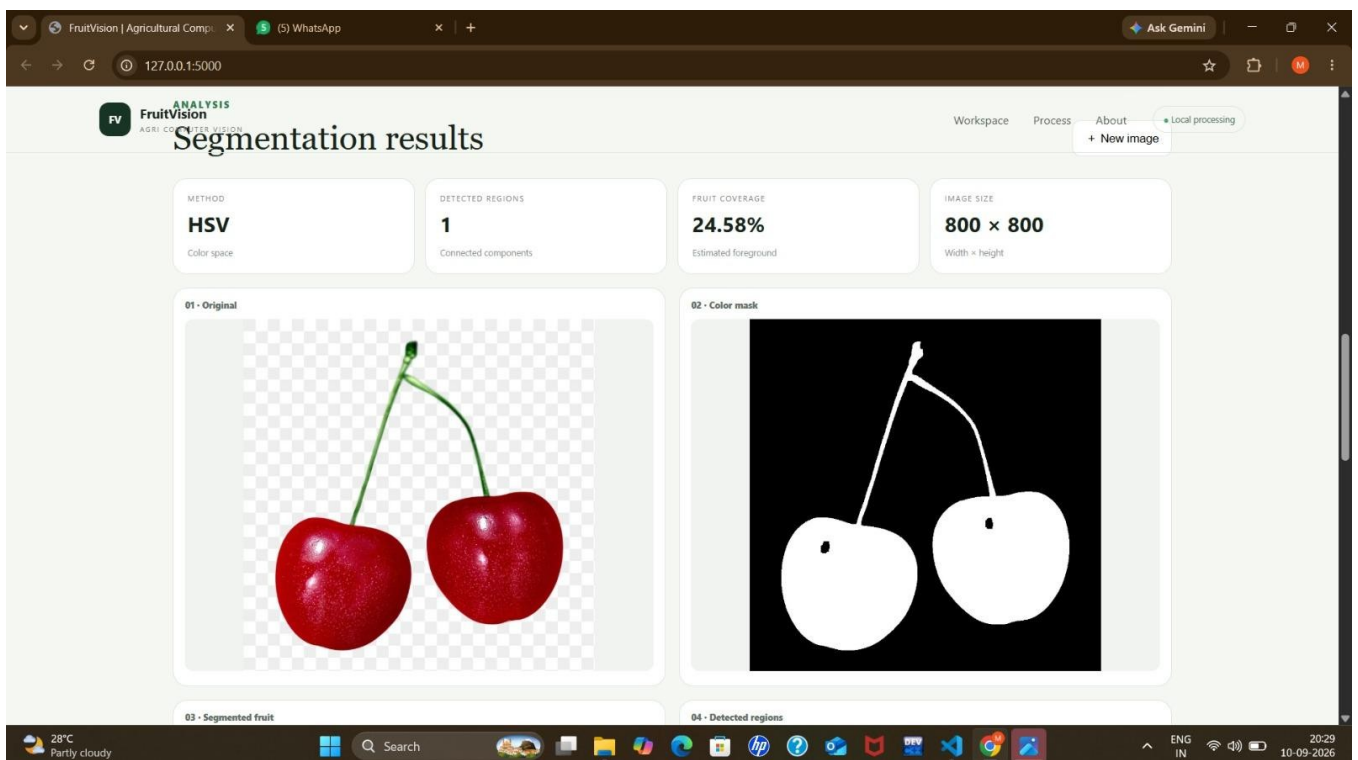
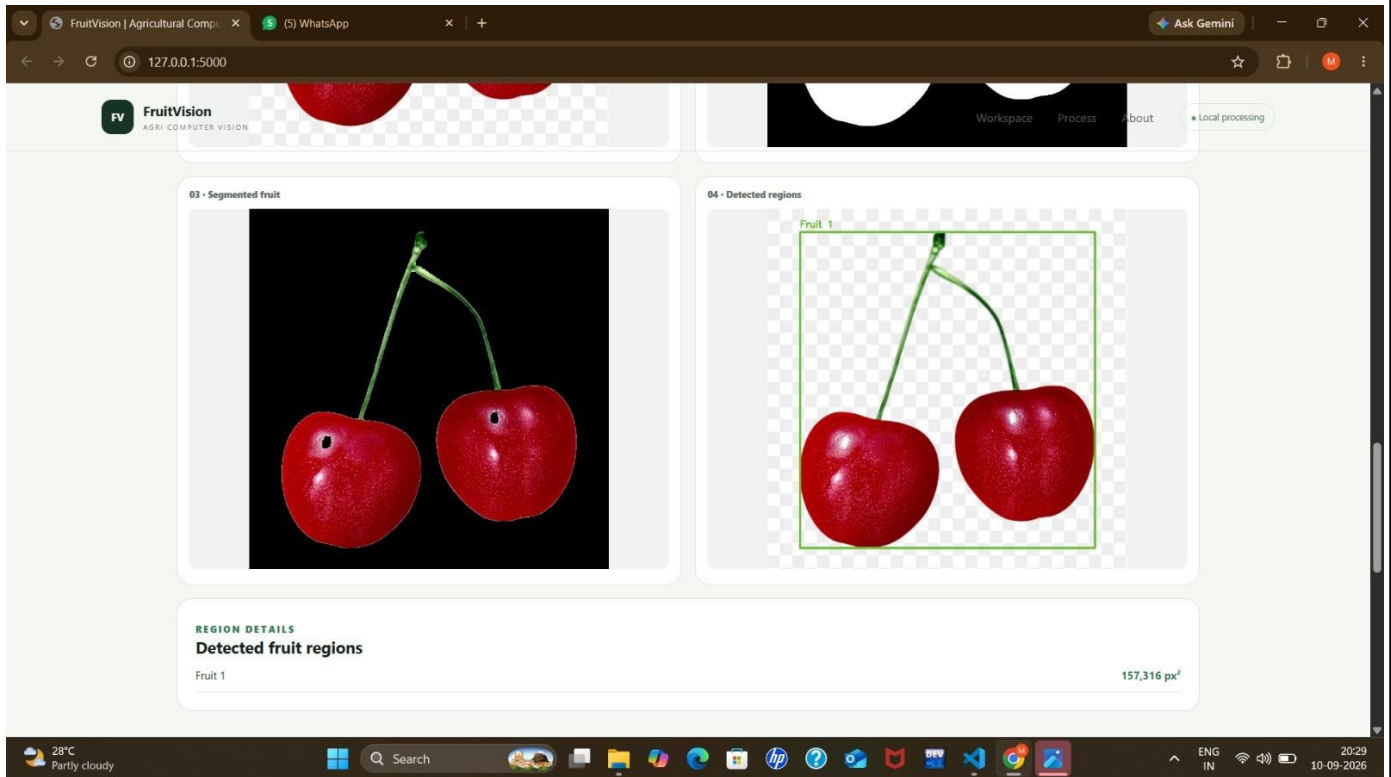
PNG, JPG or JPEG

Choose color space

HSV
Hue · Saturation · Value

LAB
Lightness · A · B

[Run segmentation --](#)



12.CONCLUSION AND FUTURE SCOPE

In this project, we implemented, evaluated, and deployed Color-Based Segmentation for Fruit Classification in Agriculture. The system uses HSV/LAB color spaces, color thresholding, binary masking, and morphological operations to effectively separate fruit regions from their background. The segmented fruit regions were further analyzed using color features for fruit classification. The interactive web application provides a hands-on environment for uploading fruit images, performing segmentation, and observing the classification results. The proposed approach is simple, efficient, and useful for applications such as automated fruit sorting, quality inspection, ripeness detection, and smart agriculture.

Future Work

1. **Deep Learning Integration:** Integrating CNN-based models to improve fruit classification accuracy and handle complex fruit images.
2. **Real-Time Detection:** Using cameras and real-time image processing for continuous fruit detection and classification.
3. **Automatic Fruit Sorting:** Integrating the system with conveyor belts and robotic mechanisms for automated sorting of different fruits.
4. **Ripeness and Disease Detection:** Extending the system to identify fruit ripeness levels and detect diseases or surface defects.
5. **Advanced Segmentation:** Using adaptive thresholding and advanced segmentation techniques to improve performance under different lighting and background conditions.

13.REFERENCES

- Gonzalez, R. C., & Woods, R. E., "Digital Image Processing," 4th Edition, Pearson, 2018.
- OpenCV Documentation, "OpenCV: Open Source Computer Vision Library," OpenCV Documentation.
- Szeliski, R., "Computer Vision: Algorithms and Applications," 2nd Edition, Springer, 2022.
- Jain, A. K., "Fundamentals of Digital Image Processing," Prentice Hall, 1989.
- Russ, J. C., "The Image Processing Handbook," 4th Edition, CRC Press, 2002.
- Bradski, G., & Kaehler, A., "Learning OpenCV: Computer Vision with the OpenCV Library," O'Reilly Media, 2008.
- Python Software Foundation, "Python Documentation," Python Programming Language Documentation.
- NumPy Developers, "NumPy Documentation," Numerical Computing Library Documentation.

14.APPENDIX

14.1. Code

```
import cv2
import numpy as np

# Read input image
image = cv2.imread("fruit.jpg")

# Convert BGR image to HSV
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

# Define color threshold
lower = np.array([0, 50, 50])
upper = np.array([30, 255, 255])

# Create binary mask
mask = cv2.inRange(hsv, lower, upper)

# Apply morphological operations
kernel = np.ones((5, 5), np.uint8)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)

# Extract fruit region
segmented = cv2.bitwise_and(image, image, mask=mask)

# Display results
cv2.imshow("Original Image", image)
cv2.imshow("Binary Mask", mask)
cv2.imshow("Segmented Fruit", segmented)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

14.2 Key Application Domains

1. Automated Fruit Sorting

Color-based segmentation can identify different fruits and assist automated sorting systems based on fruit type and appearance.

2. Fruit Quality Inspection

The system can help identify unwanted regions, color variations, and visible defects during quality checking.

3. Ripeness Detection

Changes in fruit color can be analyzed to estimate different stages of fruit ripeness.

4. Agricultural Automation

Computer vision can reduce manual effort and support automated agricultural processes.

5. Smart Farming

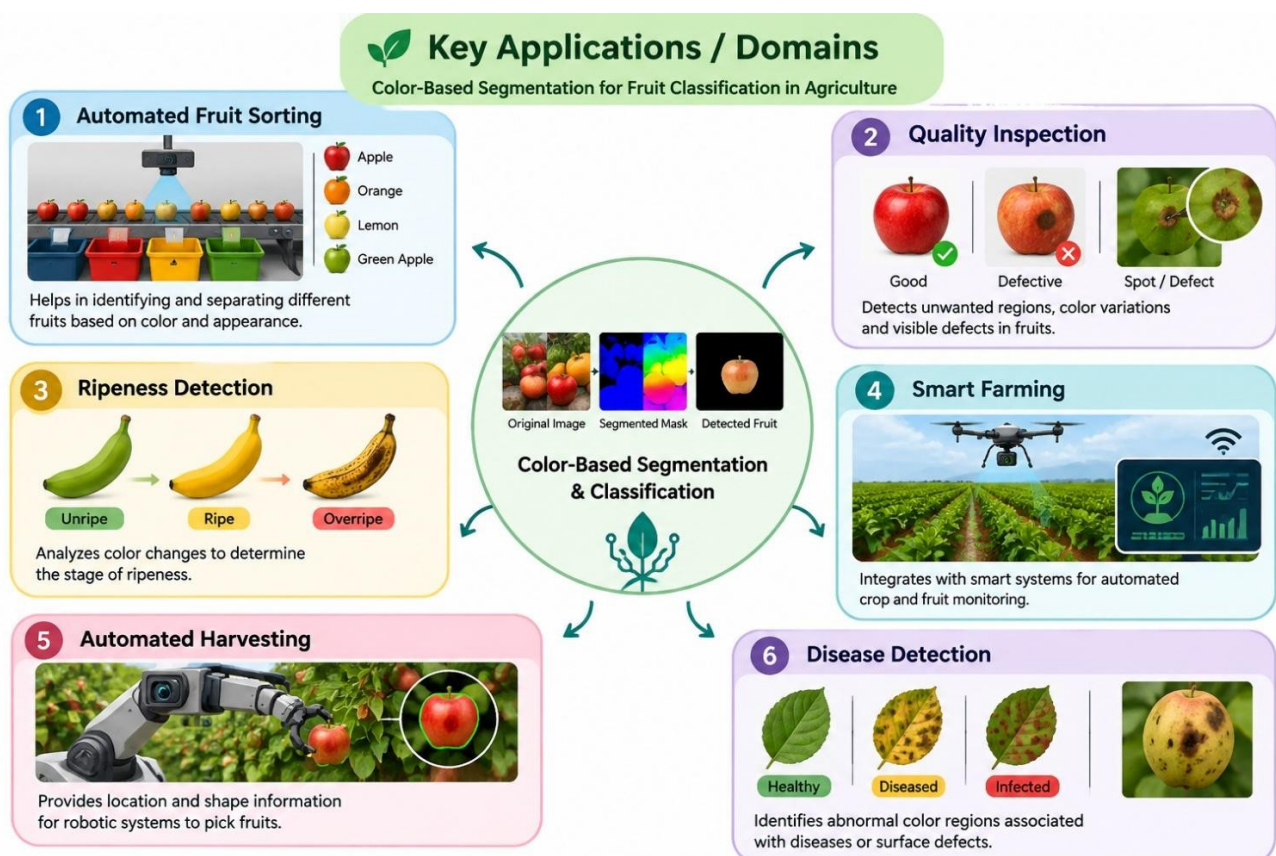
The technique can be integrated with smart farming systems for automated crop and fruit monitoring.

6. Robotic Harvesting

Segmented fruit regions can provide useful information to robotic systems for locating fruits during harvesting.

7. Fruit Disease Detection

The approach can be extended to detect abnormal color regions associated with diseases or surface defects.



Raw Data With Filters:

